Schritt 40 — Rate Limiting & GraphQL Security

In Schritt 39 schützen wir den Server vor einer zu aufwendigen GraphQL-Query.

In Schritt 40 lösen wir ein anderes Problem:

> Was passiert, wenn Client, Bot oder Angreifer zu viele GraphQL-Requests senden?

Im Rails-+GraphQL-Projekt sollte Security als mehrschichtiges System verstanden werden:

```
CLIENT
  │
  │ POST /graphql
  ▼
Rack::Attack
  │
  │ Rate Limiting
  ▼
GraphqlController
  │
  ▼
GraphQL Schema
  │
  ├── Depth Limit
  ├── Complexity Limit
  └── GraphQL Validation
  │
  ▼
Authentication
  │
  ▼
Policy / Authorization
  │
  ▼
Resolver / Mutation
  │
  ▼
Service
  │
  ▼
Database
```

1. Was ist Rate Limiting?

Rate Limiting begrenzt:

```
wie oft jemand eine API
innerhalb eines Zeitraums aufrufen darf
```

Beispiel:

```
100 requests / 1 minute / IP
```

Das bedeutet:

```
IP: 84.123.10.20

request 1    → OK
request 2    → OK
...
request 100  → OK

request 101  → BLOCK
```

HTTP response:

```
429 Too Many Requests
```

2. Warum braucht GraphQL Rate Limiting?

REST hat typischerweise viele Endpoints:

```
GET /projects
GET /tasks
POST /tasks
DELETE /tasks/10
```

GraphQL hat typischerweise nur:

```
POST /graphql
```

Innerhalb von /graphql kann jedoch stehen:

```
query {
  me {
    id
  }
}
```

oder:

```
query {
  projects {
    nodes {
      tasks {
        nodes {
          assignee {
            projects {
              nodes {
                tasks {
                  nodes {
                    id
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

Beides ist:

```
POST /graphql
```

aber die Kosten sind völlig unterschiedlich.

Deshalb braucht GraphQL mindestens zwei Schutzrichtungen:

```
Rate Limiting
→ gegen zu viele Requests

Complexity / Depth
→ gegen einen zu teuren Request
```

3. Das verwendete Werkzeug

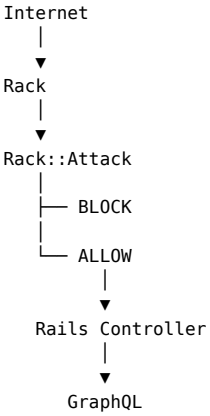
In Rails:

```
Rack::Attack
```

Es liegt vor:

```
GraphqlController
```

Flow:



Das ist sehr wichtig:

> Ein rate-limitierter Request wird blockiert, bevor Resolver, Service und SQL ausgeführt werden.

4. rack-attack installieren

Öffne:

```
Gemfile
```

hinzufügen:

```
gem "rack-attack"
```

Danach:

```
bundle install
```

5. Middleware prüfen

Ausführen:

```
bin/rails middleware
```

Du solltest sehen:

```
Rack::Attack
```

im Middleware-Stack.

6. Initializer erstellen

Erstelle genau diese Datei:

```
config/initializers/rack_attack.rb
```

Zunächst:

```
# frozen_string_literal: true

class Rack::Attack
end
```

Nun fügen wir die Schutzschichten einzeln hinzu.

7. Erste Regel: /graphql nach IP

In:

```
config/initializers/rack_attack.rb
```

hinzufügen:

```
class Rack::Attack
  throttle(
    "graphql/ip",
    limit: 100,
    period: 1.minute
  ) do |request|
    if request.post? &&
       request.path == "/graphql"
      request.ip
    end
  end
end
```

Bedeutung:

```
"graphql/ip"
→ Name der Regel

limit: 100
→ maximal 100 Requests

period: 1.minute
→ innerhalb einer Minute

request.ip
→ für jede IP separat zählen
```

8. Warum muss der Block request.ip zurückgeben?

Dieser Teil ist sehr wichtig.

Rack::Attack muss wissen:

> Wonach wird rate-limitiert?

Der Ausdruck:

```
request.ip
```

kann zurückgeben:

```
84.123.10.20
```

Rack::Attack erzeugt ungefähr diesen Counter:

```
graphql/ip:84.123.10.20
```

Eine andere IP:

```
92.50.20.8
```

hat einen anderen Counter:

```
graphql/ip:92.50.20.8
```

Also nicht:

```
100 Requests für die gesamte Website
```

sondern:

```
100 Requests / IP / Minute
```

9. Was passiert bei einem Request, der nicht /graphql ist?

Beispiel:

```
GET /health
```

Bedingung:

```
request.path == "/graphql"
```

ist:

```
false
```

Der Block gibt nil zurück.

Rack::Attack versteht:

```
diese Regel gilt nicht
```

10. Response bei Blockierung

Der Client soll keine unverständliche Response erhalten.

Thêm:

```
self.throttled_responder = lambda do |request|
  match_data =
    request.env["rack.attack.match_data"] || {}

  now = match_data[:epoch_time]

  retry_after =
    if now &&
       match_data[:period]
      match_data[:period] -
        (now % match_data[:period])
    end

  headers = {
    "Content-Type" => "application/json"
  }

  headers["Retry-After"] =
    retry_after.to_s if retry_after

  body = {
    error: "Too many requests",
    code: "RATE_LIMITED"
  }

  [
    429,
    headers,
    [body.to_json]
  ]
```

```
end
```

Nun erhält der Client:

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
```

Body:

```
{
  "error": "Too many requests",
  "code": "RATE_LIMITED"
}
```

11. Erste vollständige Datei

Aktuell:

```
config/initializers/rack_attack.rb
```

có thể ist:

```
# frozen_string_literal: true

class Rack::Attack
  throttle(
    "graphql/ip",
    limit: 100,
    period: 1.minute
  ) do |request|
    if request.post? &&
       request.path == "/graphql"
      request.ip
    end
  end

  self.throttled_responder = lambda do |request|
    match_data =
      request.env["rack.attack.match_data"] || {}

    now = match_data[:epoch_time]

    retry_after =
      if now &&
         match_data[:period]
        match_data[:period] -
          (now % match_data[:period])
      end

    headers = {
      "Content-Type" => "application/json"
    }

    headers["Retry-After"] =
      retry_after.to_s if retry_after

    body = {
      error: "Too many requests",
      code: "RATE_LIMITED"
    }

    [
      429,
      headers,
      [body.to_json]
    ]
  end
end
```

Damit funktioniert Rate Limiting bereits.

12. Praxistest — Limit auf 5 reduzieren

Nicht testen mit:

```
limit: 100
```

Vorübergehend ändern auf:

```
limit: 5,
period: 1.minute
```

Rails neu starten:

```
bin/rails server
```

13. GraphQL-Request senden

Beispiel:

```
query GetMe {
  me {
    id
    email
  }
}
```

Frontend:

```
const query = `
  query GetMe {
    me {
      id
      email
    }
  }
`;
```

Axios:

```
await axios.post(
  "/graphql",
  {
    query,
    operationName: "GetMe"
  },
  {
    withCredentials: true
  }
);
```

14. Button zum Spam-Test erstellen

React:

```
import axios from "axios";

export default function RateLimitTest() {
  const sendRequests = async () => {
    for (let i = 1; i <= 10; i++) {
      try {
        const response =
          await axios.post(
            "/graphql",
            {
              query: `
                query GetMe {
                  me {
                    id
                    email
                  }
                }
              `,

              operationName: "GetMe"
            },
            {
              withCredentials: true
            }
          );

        console.log(
          `Request ${i}:`,
          response.status
        );
      } catch (error) {
        console.log(
          `Request ${i}:`,
          error.response?.status
        );
      }
    }
  };
```

```
  return (
    <button onClick={sendRequests}>
      Test Rate Limit
    </button>
  );
}
```

Click:

```
Test Rate Limit
```

15. Ergebnis

Nếu limit ist:

```
limit: 5
```

Die Console zeigt ungefähr:

```
Request 1: 200
Request 2: 200
Request 3: 200
Request 4: 200
Request 5: 200

Request 6: 429
Request 7: 429
Request 8: 429
Request 9: 429
Request 10: 429
```

Das zeigt, dass Rack::Attack funktioniert.

16. Frontend muss 429 behandeln

Nicht nur:

```
console.log(error);
```

Besser:

```
try {
  const response =
    await axios.post(
      "/graphql",
      payload,
      {
        withCredentials: true
      }
    );

  return response.data;
} catch (error) {
  if (error.response?.status === 429) {
    throw new Error(
      "Too many requests. Please try again later."
    );
  }

  throw error;
}
```

Die UI kann anzeigen:

```
Too many requests.
Please try again later.
```

17. Rate Limit nur nach IP hat Probleme

Angenommen, ein Unternehmen hat 50 Mitarbeiter.

Alle verwenden dasselbe Unternehmensnetzwerk:

```
Public IP:
80.120.10.5
```

User A:

```
30 requests
```

User B:

```
30 requests
```

User C:

```
50 requests
```

Gesamt:

```
110
```

Nếu limit:

```
100/IP/minute
```

dann können alle betroffen sein.

Deshalb sollte eine authentifizierte GraphQL-API zusätzlich bedenken:

```
IP limit
+
User limit
```

18. Rate Limit nach User

Hier ist Vorsicht nötig.

Rack::Attack läuft vor dem GraphQL-Resolver und besitzt daher nicht automatisch:

```
context[:current_user]
```

Nicht annehmen, dass:

```
request.context[:current_user]
```

vorhanden ist.

Ist es nicht.

19. Wenn Authentication auf Session basiert

Nếu project Rails + Devise của bạn dùng session/cookie, việc rate-limit theo user ở middleware phụ thuộc vào cách authentication/session middleware được bố trí.

Nicht einfach so schreiben:

```
User.find(...)
```

in jedem Rack::Attack-Request.

Sehr schlechtes Beispiel:

```
throttle(...) do |request|
  User.find(
    request.params[:user_id]
  ).id
end
```

Warum?

```
request
↓
Rack::Attack
↓
SQL zum Finden des Users
```

Damit erzeugst du Datenbankarbeit direkt in der Anti-Spam-Schicht.

Angreifer spammt:

```
10,000 requests
```

→ der Server kann weiterhin zu sehr vielen SQL-Abfragen gezwungen werden.

20. Nicht anhand einer vom Client gesendeten userId rate-limiten

Auf keinen Fall:

```
request.params["userId"]
```

zur Bestimmung der Security-Identität.

Der Client kann senden:

```
{
  "userId": 1
}
```

im nächsten Request:

```
{
  "userId": 2
}
```

rồi:

```
{
  "userId": 3
}
```

So kann der Angreifer selbst den Rate-Limit-Bucket wechseln.

Die Security-Identität muss stammen aus:

```
trusted authentication
```

nicht aus GraphQL-Variables.

21. Sichere Basis für das aktuelle Projekt

Auf Rack-Ebene beginnen wir mit:

```
IP Rate Limit
```

Beispiel:

```
throttle(
  "graphql/ip",
  limit: 100,
  period: 1.minute
) do |request|
  request.ip if
    request.post? &&
    request.path == "/graphql"
end
```

Sau này khi authentication architecture rõ ràng, mới hinzufügen:

```
authenticated user limit
```

22. Mehrere Rate Limits können gleichzeitig existieren

Production-Beispiel:

```
60 requests / minute
+
1000 requests / hour
```

Warum?

Ein Client kann senden:

```
59 request/minute
```

kontinuierlich.

Er überschreitet nie:

```
60/min
```

aber innerhalb einer Stunde entstehen:

```
3540 requests
```

Deshalb kann man zwei Grenzen verwenden.

23. Zwei Throttles

```
class Rack::Attack
  throttle(
    "graphql/ip/minute",
    limit: 60,
    period: 1.minute
  ) do |request|
    request.ip if
      request.post? &&
      request.path == "/graphql"
```

```
  end

  throttle(
    "graphql/ip/hour",
    limit: 1000,
    period: 1.hour
  ) do |request|
    request.ip if
      request.post? &&
      request.path == "/graphql"
  end
end
```

Nun:

```
IP
│
├── max 60/minute
│
└── max 1000/hour
```

24. Praxisbeispiel

Ein User verwendet das Dashboard normal:

```
08:00 → 10 requests
08:01 → 8 requests
08:02 → 12 requests
```

Kein Problem.

Bot:

```
08:00:01 → request
08:00:01 → request
08:00:01 → request
...
```

bei:

```
request 61
```

→ 429.

25. GraphQL hat jedoch ein besonderes Problem

Angenommen, das Limit ist:

```
60 request/min
```

Der Angreifer muss keine 1000 Requests senden.

Er sendet nur:

```
50 request/min
```

aber jeder Request ist:

```
query Attack {
  projects(first: 100) {
    nodes {
      members {
        user {
          projects(first: 100) {
            nodes {
              tasks(first: 100) {
                nodes {
                  assignee {
                    projects(first: 100) {
                      nodes {
                        id
                      }
                    }
                  }
                }
              }
            }
          }
        }
      }
    }
  }
```

```
}
```

Rack::Attack sieht:

```
50 < 60
→ ALLOW
```

Die GraphQL-Query ist jedoch sehr teuer.

Genau deshalb ist Rate Limiting nicht ausreichend.

26. Schritt 39 + Schritt 40 kombinieren

Das Schema braucht einen zur graphql-ruby-Version passenden Complexity-/Depth-Schutz.

Concept:

```
Request
   │
   ▼
Rack::Attack
   │
   │ 50 requests/min
   │ OK
   ▼
GraphQL
   │
   ├── depth?
   │
   └── complexity?
   │
   ▼
Reject query n□u quá nặng
```

Beide Schichten lösen unterschiedliche Probleme.

27. Attack-Beispiel 1 — HTTP Flood

Attacker:

```
POST /graphql
POST /graphql
POST /graphql
...
10,000 l□n
```

Schutz:

```
Rack::Attack
```

28. Attack số 2 — Deep Query

Der Angreifer sendet:

```
query {
  projects {
    nodes {
      tasks {
        nodes {
          project {
            tasks {
              nodes {
                project {
                  tasks {
                    nodes {
                      id
                    }
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

Schutz:

```
Depth Limit
```

29. Attack số 3 — Wide / Expensive Query

Die Query muss nicht unbedingt sehr tief sein:

```
query {
  projects(first: 100) {
    nodes {
      id
      name
      members {
        id
      }
      tasks(first: 100) {
        nodes {
          id
          title
          description
          status
          priority
        }
      }
    }
  }
}
```

Schutz:

```
Complexity Limit
+
Pagination limits
```

30. Attack số 4 — Pagination Abuse

Du hast:

```
field :tasks,
      Types::TaskType.connection_type,
      null: false
```

Client gửi:

```
query {
  tasks(first: 1000000) {
    nodes {
      id
    }
  }
}
```

Der Client sollte keine unbegrenzte Anzahl selbst bestimmen dürfen.

31. Pagination begrenzen

Beispiel:

```
field :tasks,
      Types::TaskType.connection_type,
      null: false,
      max_page_size: 100
```

Nun kann der Client nicht regulär abrufen:

```
1,000,000 records
```

in einer Page.

Concept:

```
Rate Limit
→ Anzahl der Requests

Complexity
→ Kosten der Query

Depth
→ Tiefe

max_page_size
→ Anzahl Records/Page
```

32. Attack số 5 — Mutation Spam

Rate Limiting gilt nicht nur für Queries.

Beispiel-Mutation:

```
mutation {
  createTask(
    input: {
      projectId: "10"
      attributes: {
        title: "spam"
      }
    }
  ) {
    task {
      id
    }
  }
}
```

Der Bot sendet:

```
1000 createTask mutations
```

Wenn /graphql nur allgemein rate-limitiert wird:

```
query
mutation
```

werden beide gezählt.

Das hilft bereits deutlich.

33. Sensible Mutations können strengere Limits benötigen

Beispiel:

```
login
password reset
invite member
send email
create export
```

Diese Operationen können eigene Limits benötigen.

Problem:

Rack::Attack sieht den HTTP-Request vor der GraphQL-Ausführung.

Das GraphQL-Dokument sollte für Production-Security nicht mit Regex wie folgt geparst werden:

```
request.body.read.include?(
  "createTask"
)
```

Das ist fehleranfällig und macht die Rack-Schicht unnötig komplex.

34. Operation-Level Rate Limit

Falls benötigt:

```
createTask
→ 20/minute

updateTask
→ 60/minute

dashboard
→ 100/minute
```

sollte Rate Limiting auf einer Ebene entworfen werden, die Folgendes versteht:

```
authenticated user
+
GraphQL operation
```

oder eine passende Infrastruktur/API-Gateway verwenden.

Rack::Attack sollte nicht zu einem zweiten GraphQL-Parser werden.

35. Authentication bleibt Pflicht

Rate Limit ersetzt nicht:

```
authenticate_user!
```

Beispiel:

```
def resolve(**args)
  authenticate_user!

  ...
end
```

Das sind unterschiedliche Dinge:

```
Rack::Attack
→ wie oft wird aufgerufen?

Authentication
→ wer bist du?
```

36. Authorization bleibt Pflicht

Beispiel: Der Angreifer ist authentifiziert.

Er ruft auf:

```
mutation {
  deleteProject(
    input: {
      id: "999"
    }
  ) {
    errors
  }
}
```

Das Rate Limit sagt:

```
request count OK
```

Das bedeutet aber nicht:

```
der User darf Project 999 löschen
```

Die Policy muss prüfen:

```
authorize!(project, :destroy)
```

Unterscheidung:

```
Rate Limiting
→ Frequenz

Authentication
→ Identität

Authorization
→ Berechtigung
```

37. Variables sind niemals Authorization

Client gửi:

```
{
  "isAdmin": true
}
```

oder:

```
{
  "role": "admin"
}
```

haben keinen Security-Wert.

Der Server muss verwenden:

```
context[:current_user]
```

und die Policy entscheidet über die Berechtigung.

38. Error Handling: HTTP vs GraphQL Error

Đây là điểm rất quan trọng.

Wenn Rack::Attack blockiert:

```
HTTP 429
```

Die Response kann sein:

```
{
  "error": "Too many requests",
  "code": "RATE_LIMITED"
}
```

Das ist kein GraphQL Execution Error.

Der Request hat die GraphQL-Ausführung noch nicht erreicht.

Dagegen:

```
raise GraphQL::ExecutionError,
      "Forbidden"
```

liefert typischerweise eine GraphQL-Response:

```
{
  "errors": [
    {
      "message": "Forbidden"
    }
  ]
}
```

Das sind zwei völlig unterschiedliche Ebenen.

39. Frontend muss unterscheiden

Axios:

```
try {
  const response =
    await axios.post(
      "/graphql",
      payload,
      {
        withCredentials: true
      }
    );

  const result = response.data;

  if (result.errors) {
    console.log(
      "GraphQL error:",
      result.errors
    );
  }

  return result;
} catch (error) {
  if (error.response?.status === 429) {
    console.log(
      "Rate limited"
    );

    return;
  }

  throw error;
}
```

Flow:

```
HTTP 429
→ Axios catch

HTTP 200 + GraphQL errors
→ response.data.errors
```

40. Ein praktischer Axios-Helper

Erstellen:

src/api/graphqlClient.js

Beispiel:

```
import axios from "axios";

const graphqlClient =
  axios.create({
    baseURL: "/graphql",
    withCredentials: true,
    headers: {
      "Content-Type": "application/json"
    }
  });

export async function graphqlRequest({
  query,
  variables = {},
  operationName
}) {
  try {
    const response =
      await graphqlClient.post(
        "",
        {
          query,
          variables,
          operationName
        }
      );

    if (response.data.errors) {
      throw {
        type: "GRAPHQL_ERROR",
        errors: response.data.errors
      };
    }

    return response.data.data;
  } catch (error) {
    if (error.response?.status === 429) {
      throw {
        type: "RATE_LIMITED",
        message:
          "Too many requests. Please try again later."
      };
    }

    throw error;
  }
}
```

Danach muss die Komponente Axios nicht jedes Mal neu schreiben.

41. Helper verwenden

```
const GET_ME = `
  query GetMe {
    me {
      id
      email
    }
  }
`;
```

Aufruf:

```
try {
  const data =
    await graphqlRequest({
      query: GET_ME,
      operationName: "GetMe"
    });

  console.log(data.me);
} catch (error) {
  if (
    error.type ===
    "RATE_LIMITED"
  ) {
    setError(error.message);
```

```
  }
}
```

42. Logging für Rack::Attack hinzufügen

Du solltest wissen, wann der Server wen blockiert.

Je nach Version/Konfiguration des Gems kann man Rack::Attack-Notifications abonnieren.

Concept:

```
throttled request
↓
log
↓
IP
path
time
rule
```

Beispiel für ein gewünschtes Log:

```
[Rack::Attack]
rule=graphql/ip/minute
ip=84.123.10.20
path=/graphql
status=throttled
```

Nicht loggen:

```
password
token
session cookie
Authorization header
```

43. Reverse Proxy ist sehr wichtig

Production ist häufig nicht:

```
Browser
↓
Rails
```

sondern kann sein:

```
Browser
↓
Cloudflare
↓
Load Balancer
↓
Nginx
↓
Rails
```

Bei falscher Proxy-Konfiguration:

```
request.ip
```

kann dies den Client anders repräsentieren als erwartet.

Deshalb beim Deployment prüfen:

```
trusted proxies
forwarded headers
request.ip
```

Nicht jedem X-Forwarded-For aus dem Internet blind vertrauen.

44. request.ip testen

In Development kann vorübergehend geloggt werden:

```
Rails.logger.info(
  "Request IP: #{request.ip}"
)
```

In Production muss geprüft werden:

```
User A
→ sieht Rails die richtige Client-IP?
```

```
User B
→ ist die IP plausibel?

Oder erhalten alle die IP des Proxys?
```

Nếu tất cả thành:

```
10.0.0.5
```

dann verwenden alle unbeabsichtigt denselben IP-Rate-Limit-Bucket.

45. Redis und mehrere Rails-Instanzen

Das ist ein wichtiges Production-Thema.

Angenommen, du deployest:

```
Load Balancer
     │
  ┌──┴──┐
  ▼     ▼
Rails 1  Rails 2
```

Wenn der Counter nur lokal pro Prozess liegt:

```
Rails 1
→ count 40

Rails 2
→ count 40
```

dann hat das Gesamtsystem tatsächlich erhalten:

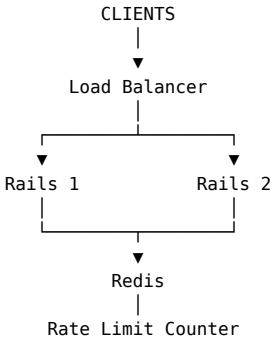
```
80
```

aber jede Instanz sieht nur:

```
40
```

Deshalb benötigt Multi-Instance-Production meist einen passenden Shared Cache, häufig Redis.

46. Architektur



So teilen alle Rails-Instanzen denselben Counter.

47. Praktisches Production-Beispiel für das Projekt

Angenommen, deine Project-Management-API hat:

```
me
projects
project
tasks
dashboard

createProject
updateProject
deleteProject

createTask
updateTask
deleteTask

addProjectMember
removeProjectMember
```

Một baseline có thể ist:

```
GraphQL endpoint
60 requests/min/IP

GraphQL endpoint
1000 requests/hour/IP

GraphQL depth
sinnvolles Limit

GraphQL complexity
sinnvolles Limit

Pagination
max 100/page

Authentication
Devise/current_user

Authorization
Policy

N+1
Dataloader
```

Die Rate-Limit-Zahlen hier sind Lern-/Testbeispiele, nicht die richtige Production-Konfiguration für jede Anwendung.

48. Vollständige Rack::Attack-Datei für die Lektion

```
# frozen_string_literal: true

class Rack::Attack
  GRAPHQL_PATH = "/graphql"

  throttle(
    "graphql/ip/minute",
    limit: 60,
    period: 1.minute
  ) do |request|
    request.ip if
      request.post? &&
      request.path == GRAPHQL_PATH
  end

  throttle(
    "graphql/ip/hour",
    limit: 1_000,
    period: 1.hour
  ) do |request|
    request.ip if
      request.post? &&
      request.path == GRAPHQL_PATH
  end

  self.throttled_responder = lambda do |request|
    match_data =
      request.env["rack.attack.match_data"] || {}

    period =
      match_data[:period]

    now =
      match_data[:epoch_time]

    retry_after =
      if period && now
        period - (now % period)
      end

    headers = {
      "Content-Type" =>
        "application/json"
    }

    headers["Retry-After"] =
      retry_after.to_s if retry_after

    body = {
```

```
      error: "Too many requests",
      code: "RATE_LIMITED"
    }

    [
      429,
      headers,
      [body.to_json]
    ]
  end
end
```

Pfad:

```
config/
└── initializers/
    └── rack_attack.rb
```

49. Beim Lernen zunächst einfach beginnen

Zunächst nur verwenden:

```
throttle(
  "graphql/ip",
  limit: 5,
  period: 1.minute
) do |request|
  request.ip if
    request.post? &&
    request.path == "/graphql"
end
```

Test-Ergebnis:

```
1 → 200
2 → 200
3 → 200
4 → 200
5 → 200
6 → 429
```

Erst nach dem Verständnis erweitern auf:

```
minute limit
+
hour limit
+
shared cache
+
monitoring
```

Nicht zehn Schichten gleichzeitig konfigurieren und danach nicht wissen, welche den Request blockiert.

50. Ein vollständiges Praxis-Szenario

Angenommen, der User öffnet das Dashboard.

Das Frontend ruft auf:

```
query Dashboard {
  dashboard {
    projectsCount
    tasksCount
    completedTasksCount
  }
}
```

Request:

```
POST /graphql
```

Normaler Request

```
Browser
   │
   ▼
Rack::Attack
   │
   │ 12/60 requests
   ▼
ALLOW
```

```
   |
   ▼
GraphQL
   |
   | depth OK
   | complexity OK
   ▼
authenticate_user!
   |
   ▼
current_user
   |
   ▼
Dashboard Resolver
   |
   ▼
SQL
   |
   ▼
200
```

User-Doppelklick/Spam oder Frontend-Bug

Beispiel: React useEffect läuft in einer Schleife:

```
useEffect(() => {
  loadDashboard();
});
```

Es fehlt:

```
[]
```

Component render:

```
render
→ request
→ setState
→ render
→ request
→ setState
→ render
→ request
...
```

Das muss kein Angreifer sein.

Es kann einfach ein Frontend-Bug sein.

Nach 60 Requests:

```
Rack::Attack
→ 429
```

Der Server ist geschützt.

Das ist ein sehr praktisches Beispiel für den Nutzen von Rate Limiting.

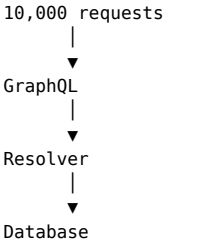
51. Ein praktisches Security-Attack-Beispiel

Der Angreifer meldet sich mit einem gültigen Account an.

Danach schreibt er ein Skript:

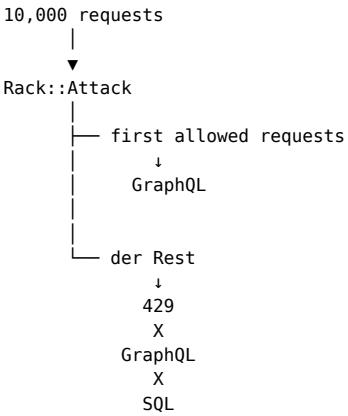
```
for (let i = 0; i < 10000; i++) {
  axios.post(
    "/graphql",
    {
      query: `
        query Dashboard {
          dashboard {
            projectsCount
            tasksCount
          }
        }
      `
    }
  );
}
```

Ohne Schutz:



Die Datenbank trägt die Last.

Mit Rack::Attack:



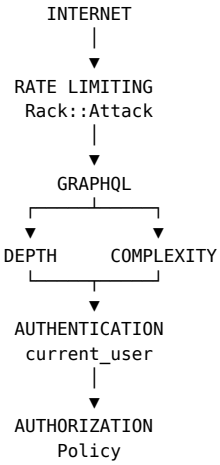
Das ist der praktische Wert von Schritt 40.

52. Übersicht der Security-Mechanismen

Problem	Protection
10.000 HTTP requests	Rate Limit
Query zu tief verschachtelt	Depth Limit
Query zu teuer	Complexity Limit
`first: 1000000`	Pagination limit
Nicht eingeloggt	Authentication
User löscht fremdes Project	Authorization / Policy
N+1 queries	Dataloader
Input mit falschem Typ	GraphQL Types / Scalars
Ungültiger Mutation-Input	Model / Service validation
Bot spammt `/graphql`	Rack::Attack

Kein einzelner Mechanismus löst alles.

53. Reihenfolge zum Merken



```
          |
          ▼
BUSINESS LOGIC
     Service
          |
          ▼
      DATABASE
```

54. Wie unterscheiden sich Schritt 39 und Schritt 40?

Bước 39

Frage:

> Ist diese Query zu komplex?

```
1 request
↓
sehr große Query
↓
BLOCK
```

Bước 40

Frage:

> Sendet dieser Client zu viele Requests?

```
1000 requests
↓
Rack::Attack
↓
BLOCK
```

Production benötigt beides.

55. Checkliste Schritt 40 für das Projekt

In dieser Reihenfolge vorgehen:

1. gem "rack-attack" hinzufügen.
2. bundle install ausführen.
3. config/initializers/rack_attack.rb erstellen.
4. POST /graphql nach IP rate-limiten.
5. Zum Test vorübergehend limit: 5 setzen.
6. Rails neu starten.
7. Mit einem React-Button 10 Requests senden.
8. Prüfen, dass Request 6 HTTP 429 erhält.
9. Frontend behandelt 429.
10. Nach dem Test auf ein passendes Production-Limit wechseln.
11. Depth- + Complexity-Schutz aus Schritt 39 beibehalten.
12. Pagination begrenzen.
13. Authentication + Policy beibehalten.
14. Beim Deployment hinter einem Proxy request.ip prüfen.
15. Bei mehreren Rails-Instanzen einen passenden Shared Cache verwenden.

Merksätze zu Schritt 40

```
Rack::Attack
→ schützt den HTTP-Endpoint

Rate Limiting
→ giới hạn Anzahl der Requests

Depth Limit
```

```
→ giới hạn Tiefe GraphQL

Complexity Limit
→ begrenzt die GraphQL-Kosten

Pagination Limit
→ begrenzt die Anzahl der Records

Authentication
→ identifiziert den User

Authorization
→ bestimmt, was der User tun darf
```

Für dein Projekt kann man sich einen Production-Request so vorstellen:

```
Axios
  ↓
POST /graphql
  ↓
Rack::Attack
  ↓
Depth / Complexity
  ↓
authenticate_user!
  ↓
current_user
  ↓
Policy
  ↓
Resolver / Mutation
  ↓
Service
  ↓
ActiveRecord
  ↓
PostgreSQL
```

Kernziel von Schritt 40: Ein Request darf nicht allein deshalb Server-Ressourcen verbrauchen, weil ein Client ihn senden kann. Rate Limit blockiert die Menge, Complexity/Depth blockiert die Kosten, und Policy blockiert den Zugriff.